



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Antispam

Curso: SI784 – Calidad y Pruebas de Software

Integrantes:

- Jahuira Pilco, Dayan Elvis (2022075749)
- Mamani Cori, Cristhian Carlos (2023077282)

Tacna – Perú

2026

Sistema Antispam **Estándar de Programación** Versión 1.0

## CONTROL DE VERSIONES

| Versión | Hecha por    | Fecha      | Motivo                                                                            |
|---------|--------------|------------|-----------------------------------------------------------------------------------|
| 1.0     | Cristhian M. | 25/06/2026 | Versión original — formaliza las convenciones ya aplicadas en el código existente |

## 1. OBJETIVO

Documentar las convenciones de codificación que rigen el desarrollo de Aegis Filter, de forma que el código nuevo (propio o de futuros colaboradores) sea consistente con el ya existente en el repositorio. Este estándar no introduce reglas nuevas: **describe lo que el código ya hace**, para que sirva como referencia verificable.

## 2. ESTÁNDAR PHP / LARAVEL (Aegis Core)

### 2.1. Base normativa

Se sigue **PSR-12** (Extended Coding Style), aplicado automáticamente mediante **Laravel Pint** ( `laravel/pint` , configuración por defecto — sin `pint.json` propio, se usa el preset `laravel` ).

```
./vendor/bin/pint           # formatea el código
./vendor/bin/pint --test    # solo verifica, no modifica (usar en CI)
```

### 2.2. Convenciones de nombres

| Elemento | Convención | Ejemplo real en el repo |
|----------|------------|-------------------------|
|----------|------------|-------------------------|

|                                         |                                                                     |                                                            |
|-----------------------------------------|---------------------------------------------------------------------|------------------------------------------------------------|
| Clases (Modelos, Controllers, Services) | PascalCase, sustantivo singular                                     | CommentController, SpamFilterService, BlacklistWord        |
| Métodos y funciones                     | camelCase, verbo de acción                                          | analyze(), checkBlacklistedWords(), findActiveByPlainKey() |
| Propiedades y variables                 | camelCase                                                           | \$maxAllowedUrls, \$keyHash                                |
| Constantes de clase                     | UPPER_SNAKE_CASE                                                    | BlacklistWord::CACHE_KEY                                   |
| Columnas de base de datos               | snake_case                                                          | created_by, is_active, spam_reason                         |
| Tablas                                  | snake_case, plural                                                  | blacklist_words, integration_keys                          |
| Rutas (URI)                             | kebab-case                                                          | /admin/integration-keys, /api/check-spam                   |
| Enums                                   | PascalCase CON CASOS PascalCase y valor string snake_case/lowercase | Channel::Wordpress = 'wordpress'                           |

### 2.3. Organización de carpetas ( src/app/ )

- Models/ — una clase Eloquent por archivo, nombre singular igual a la clase.
- Http/Controllers/ — agrupados por área cuando aplica ( Admin/ , Auth/ ); controladores de API y web conviven en el mismo namespace, diferenciados por la ruta que los invoca.
- Http/Middleware/ — un middleware por archivo, verbo Verify\* / Ensure\* cuando valida algo ( VerifyIntegrationKey ).
- Services/ — lógica de negocio reusable e inyectable, sufijo Service ( SpamFilterService , TelegramBotService ).
- Enums/ — enums nativos de PHP 8.1+, no clases de constantes.

### 2.4. Inyección de dependencias

Los servicios se inyectan por constructor (no se usa new directo dentro de los controladores ni Service Locator):

```
public function __construct(private SpamFilterService $spamFilter) {}
```

### 2.5. Comentarios

- Los modelos y migraciones llevan un bloque de comentario corto explicando el propósito de la tabla cuando no es evidente (ver 2024\_01\_01\_000001\_create\_comments\_table.php ).
- No se documenta el qué cuando el nombre ya lo dice; los comentarios explican el por qué (ej. el comentario en AppServiceProvider::boot() sobre URL::forceScheme ).

## 3. ESTÁNDAR PYTHON (alexa-bridge, discord-bridge)

### 3.1. Base normativa

PEP 8, con anotaciones de tipo (type hints) en firmas de función y docstrings breves en el módulo principal.

### 3.2. Convenciones de nombres

| Elemento                                  | Convención       | Ejemplo real                                               |
|-------------------------------------------|------------------|------------------------------------------------------------|
| Funciones y variables                     | snake_case       | check_spam(), aegis_integration_key                        |
| Constantes de módulo (config por entorno) | UPPER_SNAKE_CASE | DISCORD_BOT_TOKEN, AEGIS_REQUEST_TIMEOUT                   |
| Clases                                    | PascalCase       | (no hay clases propias; se usan las de discord.py/fastapi) |

### 3.3. Configuración por variables de entorno

Todo valor que cambia entre local/producción se lee con `os.getenv("NOMBRE", "default")` al nivel de módulo, nunca hardcodeado dentro de una función.

### 3.4. Manejo de errores: *fail-open*

Ambos *bridges* siguen la misma política: si la llamada HTTP al Aegis Core falla (timeout, 5xx, conexión rechazada), se captura la excepción, se registra con `logging`, y **no se bloquea ni se borra nada** — se prefiere dejar pasar un mensaje antes que afectar la disponibilidad del canal externo.

### 3.5. Pruebas

`pytest` + `respx` (mock de HTTP) + `unittest.mock` ( `AsyncMock` / `MagicMock` para objetos asíncronos). Un archivo de test por módulo principal, en `tests/test_*.py`.

## 4. ESTÁNDAR PARA EL PLUGIN DE WORDPRESS

Sigue las [WordPress PHP Coding Standards](#) en lugar de PSR-12, porque el código corre dentro del *runtime* de WordPress y debe integrarse con sus convenciones ( `snake_case` también en nombres de función, prefijo `aegis_filter_` en funciones globales para evitar colisiones de nombres con otros plugins).

## 5. CONTROL DE VERSIONES (Git)

- **Mensajes de commit:** en español, modo imperativo, primera línea  $\leq 72$  caracteres describiendo el *qué* (ej. `Agregar integración con Discord (discord-bridge)`); el cuerpo explica el *por qué* cuando no es obvio.
- **Ramas:** `main` es la rama estable. Cambios que afectan el pipeline de CI compartido o son riesgosos se desarrollan en una rama ( `ci/...` , `docs/...` ) y se integran vía Pull Request; cambios aislados de bajo riesgo pueden ir directo a `main` con autorización explícita.
- **Archivos generados** (cobertura, `vendor/` , `node_modules/` , reportes de mutación/Semgrep/Snyk) nunca se commitean — están en `.gitignore`.

## 6. BIBLIOGRAFÍA

- PHP-FIG. (2019). *PSR-12: Extended Coding Style Guide*.
- Laravel. (2026). Laravel Pint Documentation.
- Python Software Foundation. (2026). *PEP 8 — Style Guide for Python Code*.
- WordPress. (2026). WordPress PHP Coding Standards.

## 7. WEBGRAFÍA

- PSR-12: <https://www.php-fig.org/psr/psr-12/>
- Laravel Pint: <https://laravel.com/docs/11.x/pint>
- PEP 8: <https://peps.python.org/pep-0008/>
- WordPress Coding Standards: <https://developer.wordpress.org/coding-standards/wordpress-coding-standards/php/>